浙江大学实验报告

专业：电子科学与技术
姓名：郑智瑄
学号：3230103913
日期：May 22, 2026
地点：玉泉高分子楼-203

课程名称：人工智能实验　指导老师：魏准　实验类型：设计型实验
实验名称：卷积网络与自编码器　成　绩：　签　名：

装 订 线

一、 实验 8-1：Keras 实现 U-Net 重建

1. 实验目的

本实验使用 Keras 搭建 U-Net 形式的卷积编码器-解码器，对带噪声的 MNIST 图像进行重建，输出重建图像、相对误差以及 Loss 随 epoch 的变化曲线。同时，对比输出层保留 Skip Connection 与去除该 Skip Connection 时的重建效果，并分析两种结构的差异原因。

2. 实验原理

U-Net 由编码器、桥接层和解码器组成。编码器通过卷积和池化逐步扩大感受野，解码器通过转置卷积恢复空间尺寸。与普通自编码器相比，U-Net 在同尺度的编码端和解码端之间加入拼接式 Skip Connection，使浅层纹理和位置信息可以直接传递到解码阶段。设干净图像为 x，带噪输入为 $\tilde{x}$，模型输出为 $\hat{x}$，本实验采用均方误差作为训练目标：

$$L = \frac{1}{N}\sum_{i=1}^{N} \|x_i - \hat{x}_i\|_2^2 .$$

保留输出层 Skip Connection 时，模型的最后输出写为

$$\hat{x} = F(\tilde{x}) + \tilde{x},$$

其中 $F(\tilde{x})$ 学习的是从带噪输入到干净图像的残差修正；去除该连接时，模型直接学习

$$\hat{x} = G(\tilde{x}).$$

相对误差按如下范数形式计算：

$$\mathrm{Err} = \frac{\|x - \hat{x}\|_2}{\|x\|_2}.$$

为避免输出层初始化策略影响对比，保留和去除输出 Skip Connection 的最后一层卷积均使用 `he_normal` 初始化，并统一训练 20 个 epoch。输出层采用线性激活，原因是 MSE 回归任务中线性输出不会在训练初期受 `sigmoid` 饱和影响；评估和作图时再将像素裁剪到 $[0, 1]$。

3. 代码实现

公共数据处理函数如 Listing 1 所示。代码优先读取实验 7 中已有的本地 MNIST IDX 文件；若本地样本数不足，再退回到 `tensorflow.keras.datasets.mnist`。随后将图像归一化到 $[0, 1]$，按固定随机种子加入高斯噪声，并提供相对误差与压缩比计算函数。

Listing 1: data_utils.py：完整数据读取、加噪与指标函数

```python
import gzip
import struct
from pathlib import Path

import numpy as np


def _open_idx(path):
    path = Path(path)
    if path.suffix == ".gz":
        return gzip.open(path, "rb")
    return open(path, "rb")


def _load_idx_images(path):
    with _open_idx(path) as handle:
        magic, count, rows, cols = struct.unpack(">IIII", handle.read(16))
        if magic != 2051:
            raise ValueError(f"invalid MNIST image file: {path}")
        data = np.frombuffer(handle.read(count * rows * cols), dtype=np.uint8)
    return data.reshape(count, rows, cols).astype("float32") / 255.0


def load_mnist_images(local_root=None, limit=None):
    """Load MNIST images from local IDX files, falling back to keras.datasets."""
    if local_root is None:
        local_root = Path(__file__).resolve().parents[1] / "EXP7" / "References" / "MNIST"
    local_root = Path(local_root)
    local_images = None

    candidates = [
        local_root / "t10k-images-idx3-ubyte.gz",
        local_root / "t10k-images.idx3-ubyte",
    ]
    for image_file in candidates:
        if image_file.exists():
            local_images = _load_idx_images(image_file)
            local_images = np.expand_dims(local_images, axis=-1)
            if limit is None or local_images.shape[0] >= limit:
                return local_images[:limit] if limit is not None else local_images

    try:
        from tensorflow.keras.datasets import mnist
    except ModuleNotFoundError as exc:
        if local_images is not None:
            return local_images[:limit] if limit is not None else local_images
        raise ModuleNotFoundError(
            "TensorFlow/Keras is required when local MNIST IDX files are not found."
        ) from exc

    (x_train, _), (x_test, _) = mnist.load_data()
    images = np.concatenate([x_train, x_test], axis=0).astype("float32") / 255.0
    images = np.expand_dims(images, axis=-1)
    return images[:limit] if limit is not None else images


def make_denoising_split(
    train_count=9000,
    test_count=1000,
    noise_factor=0.5,
    seed=7,
    local_root=None,
):
    total = train_count + test_count
    images = load_mnist_images(local_root=local_root, limit=total)
    if images.shape[0] < total:
        raise ValueError(f"need {total} images, got {images.shape[0]}")

    x_train = images[:train_count]
    x_test = images[train_count:total]
    rng = np.random.default_rng(seed)
    x_train_noisy = x_train + noise_factor * rng.normal(0.0, 1.0, x_train.shape)
    x_test_noisy = x_test + noise_factor * rng.normal(0.0, 1.0, x_test.shape)
    x_train_noisy = np.clip(x_train_noisy, 0.0, 1.0).astype("float32")
    x_test_noisy = np.clip(x_test_noisy, 0.0, 1.0).astype("float32")
    return x_train_noisy, x_train, x_test_noisy, x_test


def relative_errors(ground_truth, prediction):
    gt = ground_truth.reshape((ground_truth.shape[0], -1))
    pred = prediction.reshape((prediction.shape[0], -1))
    numerator = np.linalg.norm(gt - pred, axis=1)
    denominator = np.maximum(np.linalg.norm(gt, axis=1), 1e-12)
    return numerator / denominator


def compression_ratio(input_shape, compressed_shape):
    input_size = int(np.prod(input_shape))
    compressed_size = int(np.prod(compressed_shape))
    return input_size / compressed_size
```

U-Net 的完整实现如 Listing 2 所示。编码端采用两级卷积块与最大池化，桥接层使用 64 个卷积核，解码端使用两级 `Conv2DTranspose` 恢复空间尺寸，并与对应编码层进行 `Concatenate`。输出层 Skip Connection 通过 `Add` 实现；无输出 Skip Connection 时，模型使用线性卷积层直接重建图像。

Listing 2: exp8_1_unet.py：完整 U-Net 模型与输出 Skip Connection 对照

```python
import argparse
import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from data_utils import make_denoising_split, relative_errors


def require_tensorflow():
    try:
        import tensorflow as tf
    except ModuleNotFoundError as exc:
        raise SystemExit(
            "TensorFlow is not installed. "
            "Install tensorflow or tensorflow-cpu to run this script."
        ) from exc
    return tf


def conv_block(x, filters, keras, name):
    x = keras.layers.Conv2D(
        filters,
        3,
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name=f"{name}_conv1",
    )(x)
    x = keras.layers.Conv2D(
        filters,
        3,
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name=f"{name}_conv2",
    )(x)
    return x


def build_unet(
    input_shape=(28, 28, 1),
    output_skip=True,
    learning_rate=1e-3,
    output_initializer="he_normal",
    output_activation="linear",
):
    tf = require_tensorflow()
    keras = tf.keras

    inputs = keras.Input(shape=input_shape, name="noisy_input")
    conv1 = conv_block(inputs, 16, keras, "enc1")
    pool1 = keras.layers.MaxPooling2D((2, 2), name="pool1")(conv1)
    conv2 = conv_block(pool1, 32, keras, "enc2")
    pool2 = keras.layers.MaxPooling2D((2, 2), name="pool2")(conv2)

    bridge = conv_block(pool2, 64, keras, "bridge")

    up1 = keras.layers.Conv2DTranspose(
        32,
        2,
        strides=(2, 2),
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name="up1",
    )(bridge)
    merge1 = keras.layers.Concatenate(axis=-1, name="skip_merge1")([conv2, up1])
    conv3 = conv_block(merge1, 32, keras, "dec1")

    up2 = keras.layers.Conv2DTranspose(
        16,
        2,
        strides=(2, 2),
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name="up2",
```

装 订 线

```
    )(conv3)
    merge2 = keras.layers.Concatenate(axis=-1, name="skip_merge2")([conv1, up2])
    conv4 = conv_block(merge2, 16, keras, "dec2")

    if output_skip:
        reconstruction = keras.layers.Conv2D(
            1,
            1,
            padding="same",
            activation=output_activation,
            kernel_initializer=output_initializer,
            name="residual_reconstruction",
        )(conv4)
        outputs = keras.layers.Add(name="output_skip")([reconstruction, inputs])
    else:
        outputs = keras.layers.Conv2D(
            1,
            1,
            padding="same",
            activation=output_activation,
            kernel_initializer=output_initializer,
            name="direct_output",
        )(conv4)

    model_name = "unet_with_output_skip" if output_skip else "unet_no_output_skip"
    model = keras.Model(inputs, outputs, name=model_name)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mean_squared_error",
    )
    return model


def plot_history(histories, output_path):
    plt.figure(figsize=(7.2, 4.4))
    for label, history in histories.items():
        plt.plot(history["loss"], marker="o", label=f"{label} train")
        plt.plot(history["val_loss"], marker="s", linestyle="--", label=f"{label} val")
    plt.xlabel("Epoch")
    plt.ylabel("Mean squared error")
    plt.title("Experiment 8-1: U-Net Loss")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()


def plot_reconstructions(x_noisy, x_clean, predictions, output_path, count=8):
    model_count = len(predictions)
    rows = 2 + model_count
    fig, axes = plt.subplots(rows, count, figsize=(1.32 * count, 1.35 * rows))
    titles = ["Noisy", "Clean"] + list(predictions.keys())
    row_data = [x_noisy, x_clean] + [np.clip(value, 0.0, 1.0) for value in predictions.values()]

    for row, (title, images) in enumerate(zip(titles, row_data)):
        for col in range(count):
            ax = axes[row, col]
            ax.imshow(images[col, :, :, 0], cmap="gray", vmin=0, vmax=1)
            ax.set_xticks([])
            ax.set_yticks([])
            if col == 0:
                ax.set_ylabel(title, fontsize=9)
    plt.suptitle("Experiment 8-1: U-Net Reconstruction Comparison")
    plt.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()


def plot_error_distribution(errors, output_path):
    plt.figure(figsize=(6.8, 4.2))
    labels = list(errors.keys())
    values = [errors[label] for label in labels]
    plt.boxplot(values, tick_labels=labels, showmeans=True)
    plt.ylabel("Relative error")
    plt.title("Experiment 8-1: Relative Error Distribution")
    plt.grid(axis="y", alpha=0.3)
    plt.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()


def train_and_evaluate(args):
    tf = require_tensorflow()
    tf.keras.utils.set_random_seed(args.seed)

    fig_dir = Path(args.figure_dir)
    fig_dir.mkdir(parents=True, exist_ok=True)
    x_train_noisy, x_train, x_test_noisy, x_test = make_denoising_split(
        train_count=args.train_count,
        test_count=args.test_count,
        noise_factor=args.noise_factor,
        seed=args.seed,
```

```
    )

    experiments = {
        "with_skip": True,
        "no_skip": False,
    }
    histories = {}
    predictions = {}
    errors = {}
    summary = {}

    for label, output_skip in experiments.items():
        model = build_unet(
            x_train.shape[1:],
            output_skip=output_skip,
            learning_rate=args.learning_rate,
            output_initializer=args.output_initializer,
            output_activation=args.output_activation,
        )
        history = model.fit(
            x_train_noisy,
            x_train,
            epochs=args.epochs,
            batch_size=args.batch_size,
            shuffle=True,
            validation_data=(x_test_noisy, x_test),
            verbose=2,
        )
        pred = model.predict(
            x_test_noisy[: args.eval_count],
            batch_size=args.batch_size,
            verbose=0,
        )
        err = relative_errors(x_test[: args.eval_count], np.clip(pred, 0.0, 1.0))
        histories[label] = history.history
        predictions[label] = pred
        errors[label] = err
        summary[label] = {
            "final_loss": float(history.history["loss"][-1]),
            "final_val_loss": float(history.history["val_loss"][-1]),
            "mean_relative_error": float(np.mean(err)),
            "median_relative_error": float(np.median(err)),
        }

    plot_history(histories, fig_dir / "Figure_exp8_1_loss.png")
    plot_reconstructions(
        x_test_noisy,
        x_test,
        predictions,
        fig_dir / "Figure_exp8_1_reconstruction.png",
        count=min(8, args.eval_count),
    )
    plot_error_distribution(errors, fig_dir / "Figure_exp8_1_error.png")

    summary_path = fig_dir / "exp8_1_summary.json"
    summary_path.write_text(json.dumps(summary, indent=2), encoding="utf-8")

    print("Experiment 8-1 summary")
    print(json.dumps(summary, indent=2))
    print(f"figures and summary saved to {fig_dir}")
    return summary


def parse_args():
    parser = argparse.ArgumentParser(description="Experiment 8-1: Keras U-Net reconstruction.")
    parser.add_argument("--epochs", type=int, default=20)
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--train-count", type=int, default=9000)
    parser.add_argument("--test-count", type=int, default=1000)
    parser.add_argument("--eval-count", type=int, default=200)
    parser.add_argument("--noise-factor", type=float, default=0.5)
    parser.add_argument("--learning-rate", type=float, default=1e-3)
    parser.add_argument("--output-initializer", default="he_normal")
    parser.add_argument("--output-activation", default="linear")
    parser.add_argument("--seed", type=int, default=7)
    parser.add_argument(
        "--figure-dir",
        default=str(Path(__file__).resolve().parents[2] / "Lab10 实验报告" / "figures"),
    )
    return parser.parse_args()


if __name__ == "__main__":
    train_and_evaluate(parse_args())
```

装 订 线

4. 实验结果

实验环境为 Python 3.12.9、TensorFlow 2.21.0。训练参数统一设置为：随机种子 7，噪声因子 0.5，训练样本 9000 张，验证样本 1000 张，评估样本 200 张，batch size 为 64，训练 20 个 epoch，优化器为 Adam，学习率为 10^{-3}，输出卷积层初始化为 `he_normal`。

模型	最终训练 Loss	最终验证 Loss	平均相对误差	中位相对误差
保留输出层 Skip Connection	0.009914	0.011233	0.280023	0.275722
去除输出层 Skip Connection	0.008654	0.010681	0.274785	0.273263

Table 1: 实验 8-1：同初始化条件下输出层 Skip Connection 对重建效果的影响

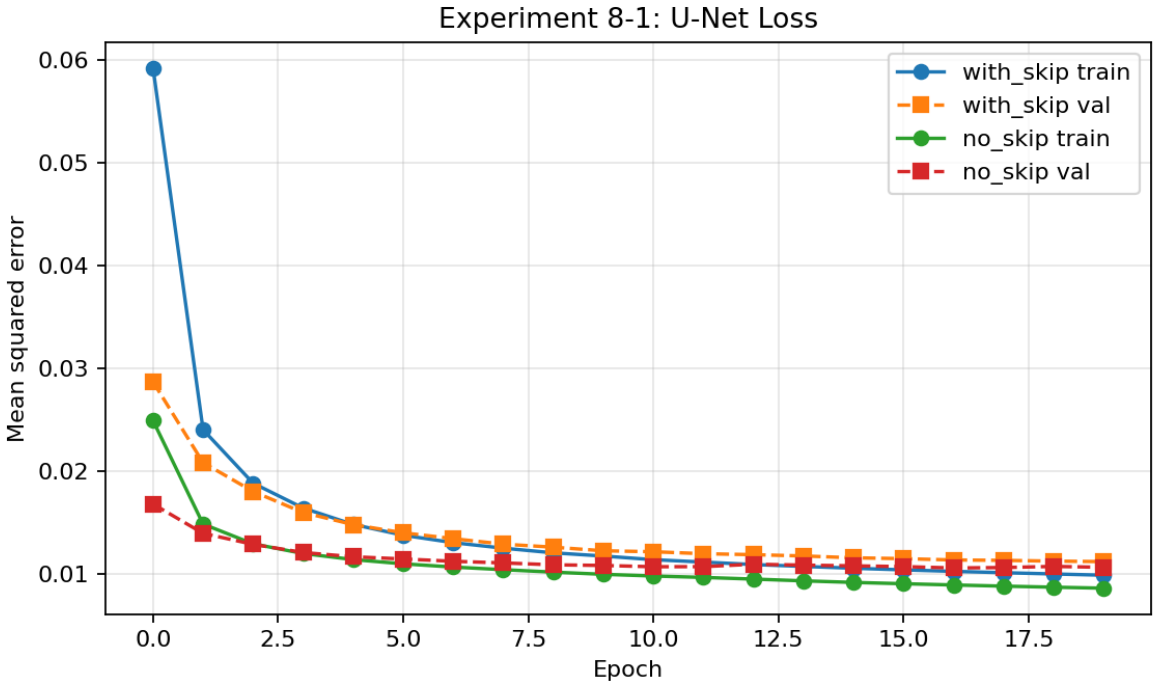


Figure 1: 实验 8-1：U-Net 训练与验证 Loss 曲线

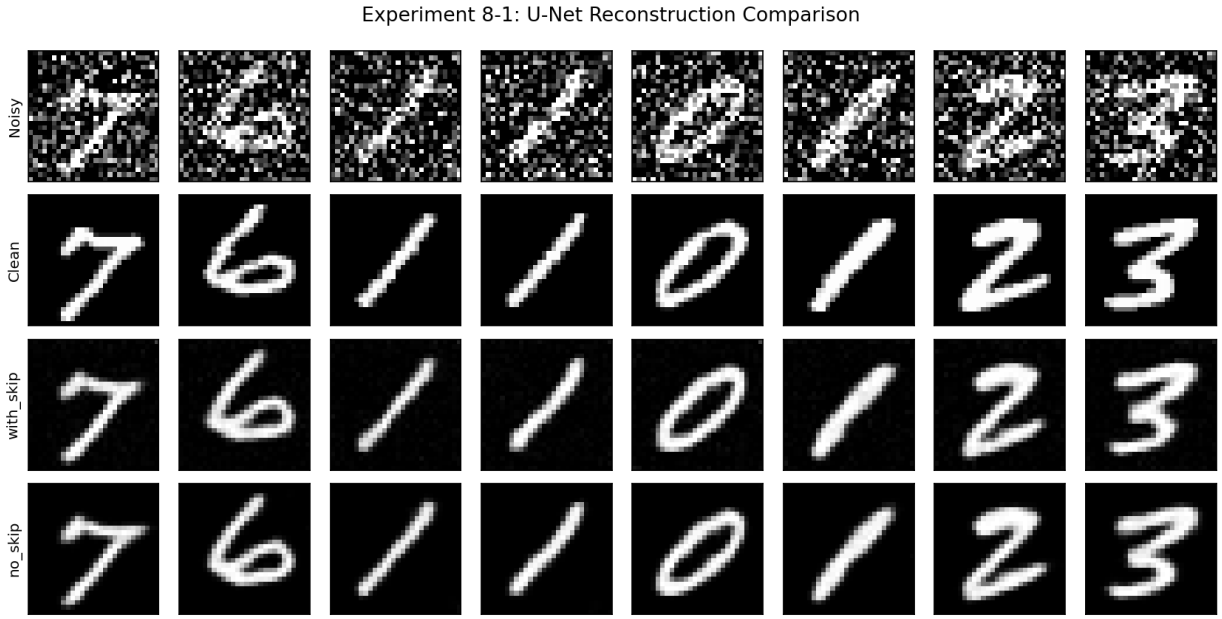


Figure 2: 实验 8-1：带噪输入、干净图像以及两种 U-Net 输出对比

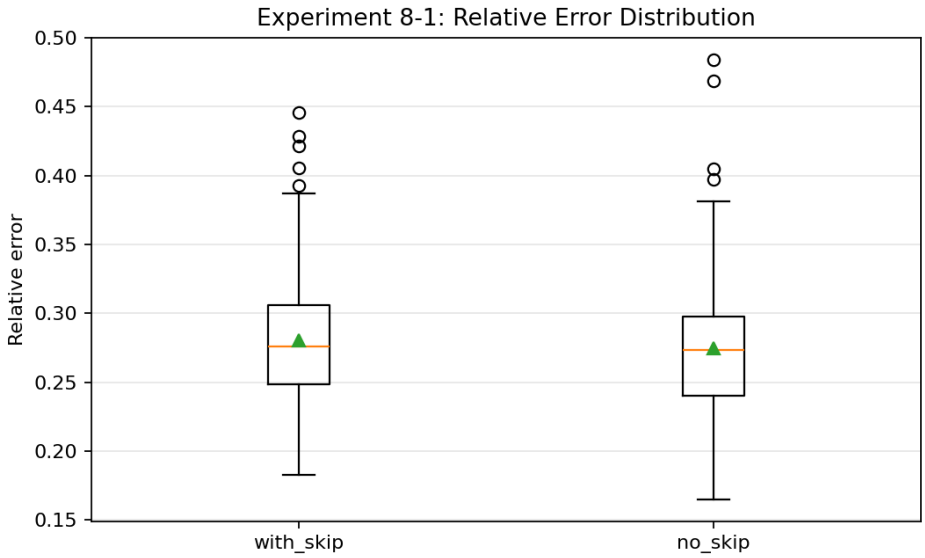


Figure 3: 实验 8-1：两种 U-Net 输出的相对误差分布

从表 1 可以看到，在统一输出层初始化并延长训练后，两种模型的差距明显缩小，去除输出层 Skip Connection 的模型仍取得略低的验证 Loss 和平均相对误差。该结论应限定在本实验设置下理解：输入 $\tilde{x}$ 本身已经含有较强噪声，输出端直接加回 $\tilde{x}$ 会显式引入一条噪声通路，模型需要学习 $x - \tilde{x}$ 这一去噪残差才能抵消它。

这一结果并不表示残差学习在图像去噪中普遍劣于直接重建。DnCNN 等经典去噪工作利用残差学习预测噪声分量，并认为噪声统计往往比自然图像内容更容易建模。图 1 中后期验证 Loss 也有轻微波动，说明继续延长训练未必总能同步改善泛化。因此，本实验的结论限定为：在当前数据规模、网络宽度、噪声强度和训练轮数下，输出端直接加回带噪输入没有带来优势；若改变初始化、网络规模或训练策略，结论仍需重新验证。

二、 实验 8-2：卷积自编码器图像去噪与压缩比搜索

1. 实验目的

本实验使用 Keras 搭建卷积神经网络自编码器，将带噪声污染的 MNIST 图像作为输入，以干净图像作为监督目标进行去噪重建。实验探索不同网络结构、优化器和损失函数，在相对误差小于 50% 的约束下，寻找尽可能大的压缩比

$$\text{Ratio} = \frac{\text{size(Input)}}{\text{size(Compressed)}}.$$

同时打印最小压缩层的 shape，并可视化去噪结果与训练过程。

2. 实验原理

卷积自编码器由编码器和解码器两部分组成。编码器通过卷积和池化把输入图像压缩为低维特征张量，解码器再通过转置卷积逐步恢复图像尺寸。本实验将压缩层命名为 `compressed`，其尺寸直接用于计算压缩比。例如输入尺寸为 $28 \times 28 \times 1$，当压缩层为 $4 \times 4 \times 2$ 时，有

$$\text{Ratio} = \frac{28 \times 28 \times 1}{4 \times 4 \times 2} = 24.5.$$

当压缩层进一步变为 $2 \times 2 \times 4$ 时，压缩比达到

$$\text{Ratio} = \frac{28 \times 28 \times 1}{2 \times 2 \times 4} = 49.$$

压缩比越大，瓶颈越窄，模型需要用更少的变量保存数字结构；但压缩过强会损失笔画细节，导致相对误差升高。

3. 代码实现

自编码器完整实现如 Listing 3 所示。候选结构覆盖两级、三级和四级池化：两级池化得到 7×7 级别的压缩层，三级池化得到 4×4 级别的压缩层，四级池化得到 2×2 级别的压缩层。三级和四级解码后空间尺寸会恢复到 32×32，因此使用 `Cropping2D` 裁剪回 28×28。候选配置包括 `d3_c2`、`d3_c1`、`d4_c4` 和 `d4_c2`，覆盖 24.5、49 和 98 倍压缩比。

Listing 3: exp8_2_autoencoder.py：完整卷积自编码器与压缩比搜索

```python
import argparse
import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from data_utils import compression_ratio, make_denoising_split, relative_errors


def require_tensorflow():
    try:
        import tensorflow as tf
    except ModuleNotFoundError as exc:
        raise SystemExit(
            "TensorFlow is not installed. "
            "Install tensorflow or tensorflow-cpu to run this script."
        ) from exc
    return tf


def optimizer_from_name(keras, name, learning_rate):
    name = name.lower()
    if name == "adam":
```

装 订 线

```
        return keras.optimizers.Adam(learning_rate=learning_rate)
    if name == "rmsprop":
        return keras.optimizers.RMSprop(learning_rate=learning_rate)
    if name == "nadam":
        return keras.optimizers.Nadam(learning_rate=learning_rate)
    if name == "sgd":
        return keras.optimizers.SGD(learning_rate=learning_rate, momentum=0.9)
    raise ValueError(f"unsupported optimizer: {name}")


def conv_block(x, filters, keras, name):
    x = keras.layers.Conv2D(
        filters,
        3,
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name=f"{name}_conv1",
    )(x)
    return keras.layers.Conv2D(
        filters,
        3,
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name=f"{name}_conv2",
    )(x)


def build_autoencoder(
    input_shape=(28, 28, 1),
    depth=2,
    latent_channels=8,
    optimizer="adam",
    loss="mean_squared_error",
    learning_rate=1e-3,
):
    tf = require_tensorflow()
    keras = tf.keras

    inputs = keras.Input(shape=input_shape, name="noisy_input")
    x = conv_block(inputs, 16, keras, "enc1")
    x = keras.layers.MaxPooling2D((2, 2), padding="same", name="pool1")(x)
    x = conv_block(x, 32, keras, "enc2")
    x = keras.layers.MaxPooling2D((2, 2), padding="same", name="pool2")(x)

    if depth >= 3:
        x = conv_block(x, 32, keras, "enc3")
        x = keras.layers.MaxPooling2D((2, 2), padding="same", name="pool3")(x)
    if depth == 4:
        x = conv_block(x, 64, keras, "enc4")
        x = keras.layers.MaxPooling2D((2, 2), padding="same", name="pool4")(x)
    elif depth not in (2, 3):
        raise ValueError("depth must be 2, 3, or 4")

    compressed = keras.layers.Conv2D(
        latent_channels,
        3,
        padding="same",
        activation="relu",
        kernel_initializer="he_normal",
        name="compressed",
    )(x)

    x = compressed
    if depth == 4:
        x = keras.layers.Conv2DTranspose(
            64,
            2,
            strides=(2, 2),
            padding="same",
            activation="relu",
            name="up4",
        )(x)
        x = conv_block(x, 64, keras, "dec4")

    if depth >= 3:
        x = keras.layers.Conv2DTranspose(
            32,
            2,
            strides=(2, 2),
            padding="same",
            activation="relu",
            name="up3",
        )(x)
        x = conv_block(x, 32, keras, "dec3")

    x = keras.layers.Conv2DTranspose(
        32,
        2,
        strides=(2, 2),
        padding="same",
        activation="relu",
```

```
        name="up2",
    )(x)
    x = conv_block(x, 32, keras, "dec2")
    x = keras.layers.Conv2DTranspose(
        16,
        2,
        strides=(2, 2),
        padding="same",
        activation="relu",
        name="up1",
    )(x)
    x = conv_block(x, 16, keras, "dec1")

    if depth >= 3:
        x = keras.layers.Cropping2D(((2, 2), (2, 2)), name="crop_to_28")(x)

    outputs = keras.layers.Conv2D(
        1,
        3,
        padding="same",
        activation="linear",
        name="clean_output",
    )(x)
    model = keras.Model(inputs, outputs, name=f"conv_autoencoder_d{depth}_c{latent_channels}")
    model.compile(
        optimizer=optimizer_from_name(keras, optimizer, learning_rate),
        loss=loss,
    )
    compressed_shape = tuple(model.get_layer("compressed").output.shape[1:])
    return model, compressed_shape


def candidate_configs():
    return [
        {
            "name": "d2_c16_adam_mse",
            "depth": 2,
            "latent_channels": 16,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d2_c8_adam_mse",
            "depth": 2,
            "latent_channels": 8,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d2_c4_adam_mse",
            "depth": 2,
            "latent_channels": 4,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c16_adam_mse",
            "depth": 3,
            "latent_channels": 16,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c8_adam_mse",
            "depth": 3,
            "latent_channels": 8,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c4_adam_mse",
            "depth": 3,
            "latent_channels": 4,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c2_adam_mse",
            "depth": 3,
            "latent_channels": 2,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c1_adam_mse",
            "depth": 3,
            "latent_channels": 1,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d4_c4_adam_mse",
            "depth": 4,
```

装 订 线

```
            "latent_channels": 4,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d4_c2_adam_mse",
            "depth": 4,
            "latent_channels": 2,
            "optimizer": "adam",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c8_rmsprop_mse",
            "depth": 3,
            "latent_channels": 8,
            "optimizer": "rmsprop",
            "loss": "mean_squared_error",
        },
        {
            "name": "d3_c8_nadam_mae",
            "depth": 3,
            "latent_channels": 8,
            "optimizer": "nadam",
            "loss": "mean_absolute_error",
        },
    ]


def plot_search(results, output_path):
    labels = [item["name"] for item in results]
    ratios = [item["compression_ratio"] for item in results]
    errors = [item["mean_relative_error"] for item in results]
    colors = ["#4c78a8" if err < 0.5 else "#e45756" for err in errors]

    fig, ax1 = plt.subplots(figsize=(10.8, 5.0))
    x_pos = np.arange(len(labels))
    ax1.bar(x_pos, ratios, color=colors, alpha=0.82)
    ax1.set_ylabel("Compression ratio")
    ax1.set_xticks(x_pos)
    ax1.set_xticklabels(labels, rotation=40, ha="right", fontsize=7)
    ax1.grid(axis="y", alpha=0.25)

    ax2 = ax1.twinx()
    ax2.plot(x_pos, errors, color="black", marker="o", linewidth=1.8)
    ax2.axhline(0.5, color="#f58518", linestyle="--", linewidth=1.2)
    ax2.set_ylabel("Mean relative error")
    ax2.set_ylim(0.0, max(0.8, max(errors) * 1.15))

    plt.title("Experiment 8-2: Compression Ratio Search")
    fig.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()


def plot_reconstructions(x_noisy, x_clean, pred, output_path, count=8):
    rows = 3
    fig, axes = plt.subplots(rows, count, figsize=(1.32 * count, 4.3))
    row_data = [x_noisy, x_clean, np.clip(pred, 0.0, 1.0)]
    titles = ["Noisy", "Clean", "Denoised"]
    for row, (title, images) in enumerate(zip(titles, row_data)):
        for col in range(count):
            ax = axes[row, col]
            ax.imshow(images[col, :, :, 0], cmap="gray", vmin=0, vmax=1)
            ax.set_xticks([])
            ax.set_yticks([])
            if col == 0:
                ax.set_ylabel(title, fontsize=9)
    plt.suptitle("Experiment 8-2: Best Autoencoder Denoising Result")
    plt.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()


def plot_loss(best_history, output_path):
    plt.figure(figsize=(6.8, 4.2))
    plt.plot(best_history["loss"], marker="o", label="train")
    plt.plot(best_history["val_loss"], marker="s", linestyle="--", label="validation")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Experiment 8-2: Best Autoencoder Loss")
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()


def train_and_search(args):
    tf = require_tensorflow()
    tf.keras.utils.set_random_seed(args.seed)

    fig_dir = Path(args.figure_dir)
    fig_dir.mkdir(parents=True, exist_ok=True)
```

装 订 线

```
    x_train_noisy, x_train, x_test_noisy, x_test = make_denoising_split(
        train_count=args.train_count,
        test_count=args.test_count,
        noise_factor=args.noise_factor,
        seed=args.seed,
    )

    results = []
    best_model = None
    best_history = None
    best_pred = None
    best_success = None

    for config in candidate_configs():
        model, compressed_shape = build_autoencoder(
            input_shape=x_train.shape[1:],
            depth=config["depth"],
            latent_channels=config["latent_channels"],
            optimizer=config["optimizer"],
            loss=config["loss"],
            learning_rate=args.learning_rate,
        )
        history = model.fit(
            x_train_noisy,
            x_train,
            epochs=args.epochs,
            batch_size=args.batch_size,
            shuffle=True,
            validation_data=(x_test_noisy, x_test),
            verbose=2,
        )
        pred = model.predict(
            x_test_noisy[: args.eval_count],
            batch_size=args.batch_size,
            verbose=0,
        )
        err = relative_errors(x_test[: args.eval_count], np.clip(pred, 0.0, 1.0))
        ratio = compression_ratio(x_train.shape[1:], compressed_shape)
        result = {
            **config,
            "compressed_shape": list(map(int, compressed_shape)),
            "compression_ratio": float(ratio),
            "final_loss": float(history.history["loss"][-1]),
            "final_val_loss": float(history.history["val_loss"][-1]),
            "mean_relative_error": float(np.mean(err)),
            "median_relative_error": float(np.median(err)),
        }
        results.append(result)

        if result["mean_relative_error"] < 0.5:
            if (
                best_success is None
                or result["compression_ratio"] > best_success["compression_ratio"]
            ):
                best_success = result
                best_model = model
                best_history = history.history
                best_pred = pred

    results = sorted(results, key=lambda item: item["compression_ratio"])
    plot_search(results, fig_dir / "Figure_exp8_2_search.png")

    if best_success is not None:
        plot_reconstructions(
            x_test_noisy,
            x_test,
            best_pred,
            fig_dir / "Figure_exp8_2_reconstruction.png",
            count=min(8, args.eval_count),
        )
        plot_loss(best_history, fig_dir / "Figure_exp8_2_loss.png")
        best_model.save(fig_dir / "exp8_2_best_autoencoder.keras")

    output = {
        "threshold": 0.5,
        "best_under_threshold": best_success,
        "all_results": results,
    }
    summary_path = fig_dir / "exp8_2_summary.json"
    summary_path.write_text(json.dumps(output, indent=2), encoding="utf-8")

    print("Experiment 8-2 compression search")
    print(json.dumps(output, indent=2))
    print(f"figures and summary saved to {fig_dir}")
    return output


def parse_args():
    parser = argparse.ArgumentParser(
        description="Experiment 8-2: Keras convolutional denoising autoencoder."
    )
    parser.add_argument("--epochs", type=int, default=20)
    parser.add_argument("--batch-size", type=int, default=64)
```

```
    parser.add_argument("--train-count", type=int, default=9000)
    parser.add_argument("--test-count", type=int, default=1000)
    parser.add_argument("--eval-count", type=int, default=200)
    parser.add_argument("--noise-factor", type=float, default=0.5)
    parser.add_argument("--learning-rate", type=float, default=1e-3)
    parser.add_argument("--seed", type=int, default=7)
    parser.add_argument(
        "--figure-dir",
        default=str(Path(__file__).resolve().parents[2] / "Lab10 实验报告" / "figures"),
    )
    return parser.parse_args()


if __name__ == "__main__":
    train_and_search(parse_args())
```

4. 实验结果

实验使用与 8-1 相同的数据划分和噪声设置，即训练样本 9000 张、验证样本 1000 张、评估样本 200 张、噪声因子 0.5、随机种子 7、batch size 64、训练 20 个 epoch。压缩比搜索结果如表 2 所示。

候选模型	压缩层 shape	压缩比	优化器	Loss	最终验证 Loss	平均相对误差	结果
d2_c16_adam_mse	7 × 7 × 16	1.0000	Adam	MSE	0.012213	0.294944	通过
d2_c8_adam_mse	7 × 7 × 8	2.0000	Adam	MSE	0.012180	0.294084	通过
d3_c16_adam_mse	4 × 4 × 16	3.0625	Adam	MSE	0.014392	0.317839	通过
d2_c4_adam_mse	7 × 7 × 4	4.0000	Adam	MSE	0.012229	0.295776	通过
d3_c8_adam_mse	4 × 4 × 8	6.1250	Adam	MSE	0.015234	0.324837	通过
d3_c8_rmsprop_mse	4 × 4 × 8	6.1250	RMSprop	MSE	0.018072	0.362297	通过
d3_c8_nadam_mae	4 × 4 × 8	6.1250	Nadam	MAE	0.136180	1.000000	失败
d3_c4_adam_mse	4 × 4 × 4	12.2500	Adam	MSE	0.016003	0.336505	通过
d3_c2_adam_mse	4 × 4 × 2	24.5000	Adam	MSE	0.020789	0.389597	通过
d3_c1_adam_mse	4 × 4 × 1	49.0000	Adam	MSE	0.070015	0.752060	失败
d4_c4_adam_mse	2 × 2 × 4	**49.0000**	Adam	MSE	0.023870	**0.407893**	**通过**
d4_c2_adam_mse	2 × 2 × 2	98.0000	Adam	MSE	0.069798	0.749671	失败

Table 2: 实验 8-2：不同自编码器结构、优化器与 Loss 的压缩比搜索结果

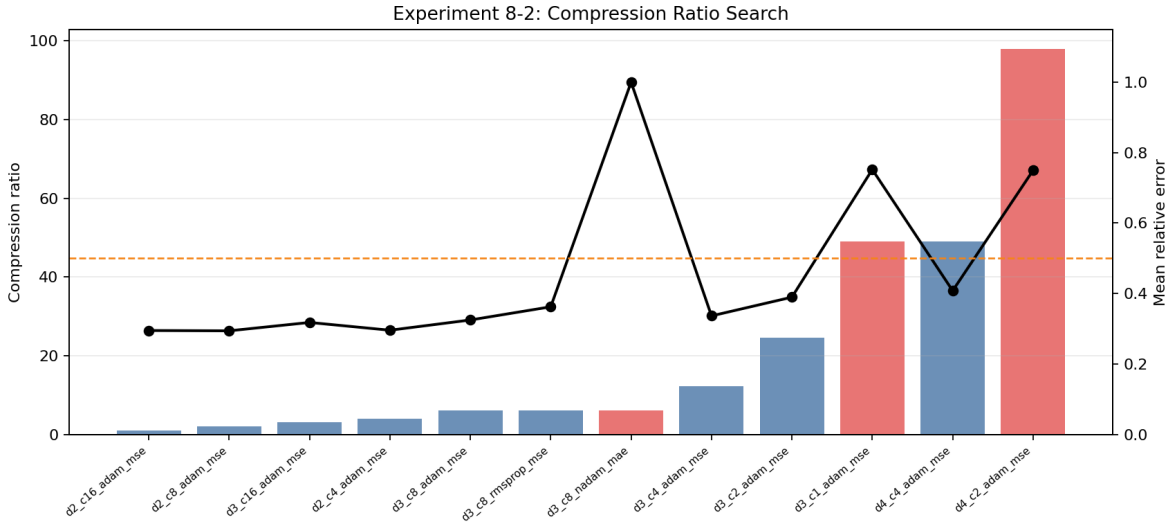


Figure 4: 实验 8-2：压缩比与平均相对误差对比，虚线为 50% 误差阈值

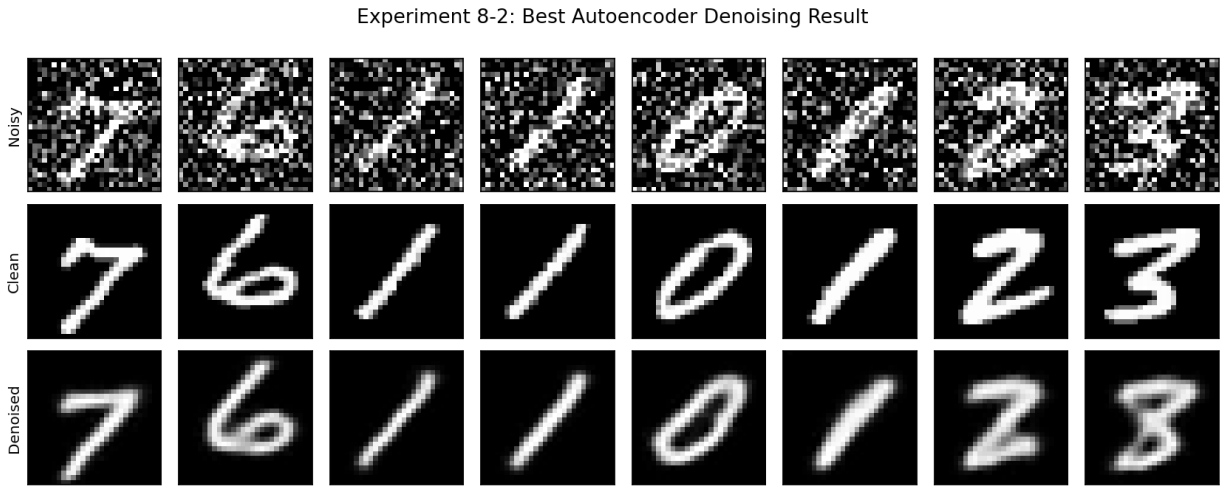


Figure 5: 实验 8-2：最优压缩比模型的去噪重建结果

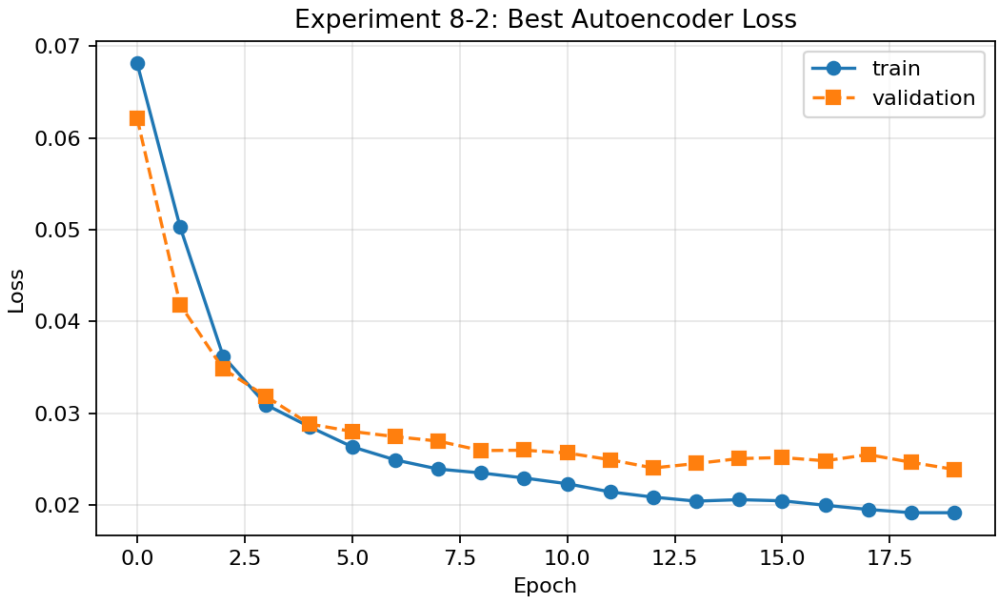


Figure 6: 实验 8-2：最优压缩比模型的训练与验证 Loss 曲线

由表 2 可知，`d3_c2_adam_mse` 在 $4\times4\times2$ 瓶颈下达到 24.5 倍压缩，平均相对误差为 0.389597；`d4_c4_adam_mse` 在 $2\times2\times4$ 瓶颈下达到 49 倍压缩，平均相对误差为 0.407893，仍低于 50% 阈值。因此，本次搜索得到的**最大可行压缩比为 49.0**，对应压缩层 shape 为 $2\times2\times4$。

继续压缩到 $2\times2\times2$ 时，平均相对误差升至 0.749671，已明显超过阈值；同为 49 倍压缩的 $4\times4\times1$ 也未达到要求，说明压缩比相同并不代表重建难度相同。`d3_c1_adam_mse` 与 `d4_c4_adam_mse` 同时改变了瓶颈形状、网络深度和解码器参数量，因此差异不能单独归因于通道数。在相同压缩比下，瓶颈空间形状、通道数和解码器规模会共同影响重建质量。

从同通道数的模型对比还可以看到，较大的空间分辨率通常带来更低误差。例如 `d2_c4_adam_mse` 的 $7\times7\times4$ 瓶颈误差为 0.295776，而 `d3_c4_adam_mse` 的 $4\times4\times4$ 瓶颈误差升至 0.336505；`d2_c8_adam_mse` 也优于

`d3_c8_adam_mse`。这说明对 MNIST 这种笔画稀疏、空间结构强的图像，保留更多空间位置有时比进一步加深压缩更有利。

图 6 中最优压缩比模型在训练后期出现轻微的 train–validation Loss 分叉：训练 Loss 继续下降，而验证 Loss 基本持平并有小幅波动。这是轻度过拟合信号，可通过加入 Dropout、weight decay 或继续扩大训练样本缓解。在完整 1000 张测试图像上，`d4_c4_adam_mse` 的平均相对误差为 0.448349，中位相对误差为 0.440049，仍低于 50% 阈值，但比 200 张评估集结果更接近边界，说明固定种子和评估样本量会带来一定浮动。

`d3_c8_nadam_mae` 的平均相对误差为 1.0，说明输出几乎退化为全黑图像。这个现象与 MAE 在大量零像素数据上的性质一致：MAE 的最优常数预测是中位数，而 MNIST 背景像素占多数且接近 0，模型在短训练中容易被推向“预测背景”的退化解。该结果反而说明，对于本去噪任务，MSE 比 MAE 更适合作为默认损失函数。

三、 实验总结

本次实验完成了两个 Keras 卷积网络任务。实验 8-1 搭建了带编码端-解码端 Skip Connection 的 U-Net，并在同输出层初始化、20 个 epoch 训练条件下重新比较输出层 Skip Connection 的影响。结果显示，无输出 Skip Connection 的直接重建分支仍略优，但差距较小；该结果只说明本实验设置下直接加回带噪输入没有明显收益，不能推导为残差学习的一般劣势。

实验 8-2 搭建并搜索了多种卷积自编码器结构。**最大可行压缩比为 49.0**，对应压缩层 $2 \times 2 \times 4$，200 张评估集上的平均相对误差为 0.407893，完整 1000 张测试图像上的平均相对误差为 0.448349。整体来看，卷积结构能够利用局部空间相关性完成有效去噪，而压缩比、瓶颈形状、网络深度、优化器和损失函数共同决定了重建质量与压缩能力之间的折中。

装 订 线